

【九析】第十二课：MCP+A2A工程协议——智能体连接的基础设施

01

课程主题

项目	内容
课程主题	MCP+A2A工程协议——大模型上下文协议详解、核心组件、生命周期管理、生态介绍与平台实操
课时安排	第8周第1次课（2小时）
学员基础	掌握Agent基础、 认知框架、 LangChain/LangGraph/OpenClaw使用、 AutoGen 多智能体协作、 CrewAI 入门与 Job Posting 实战
教学目标	1. 理解 MCP 协议的诞生背景、核心价值与技术定位 2. 掌握 MCP 的客户端-主机-服务器三层架构与核心组件 3. 理解 MCP Server 生命周期管理的完整流程与最佳实践 4. 理解 A2A 协议的定位、核心能力 5. 掌握 MCP 与 A2A 的区别与协作关系，形成完整的智能体通信协议认知 6. 完成 MCP 平台实操，体验创建 MCP Server、发布工具、从 Agent 调用工具的全流程

02

环境准备

- Python 3.11+ 环境

- OpenAI API 密钥或国产大模型 API 密钥
- 可选：AutoGen、CrewAI 或 LangGraph 的基础安装环境
- 提前安装： `pip install mcp` 或 `pip install anthropic-mcp`
- 预习：mcp 和 a2a 基本概念

03

课程详情

第一部分：开场与回顾

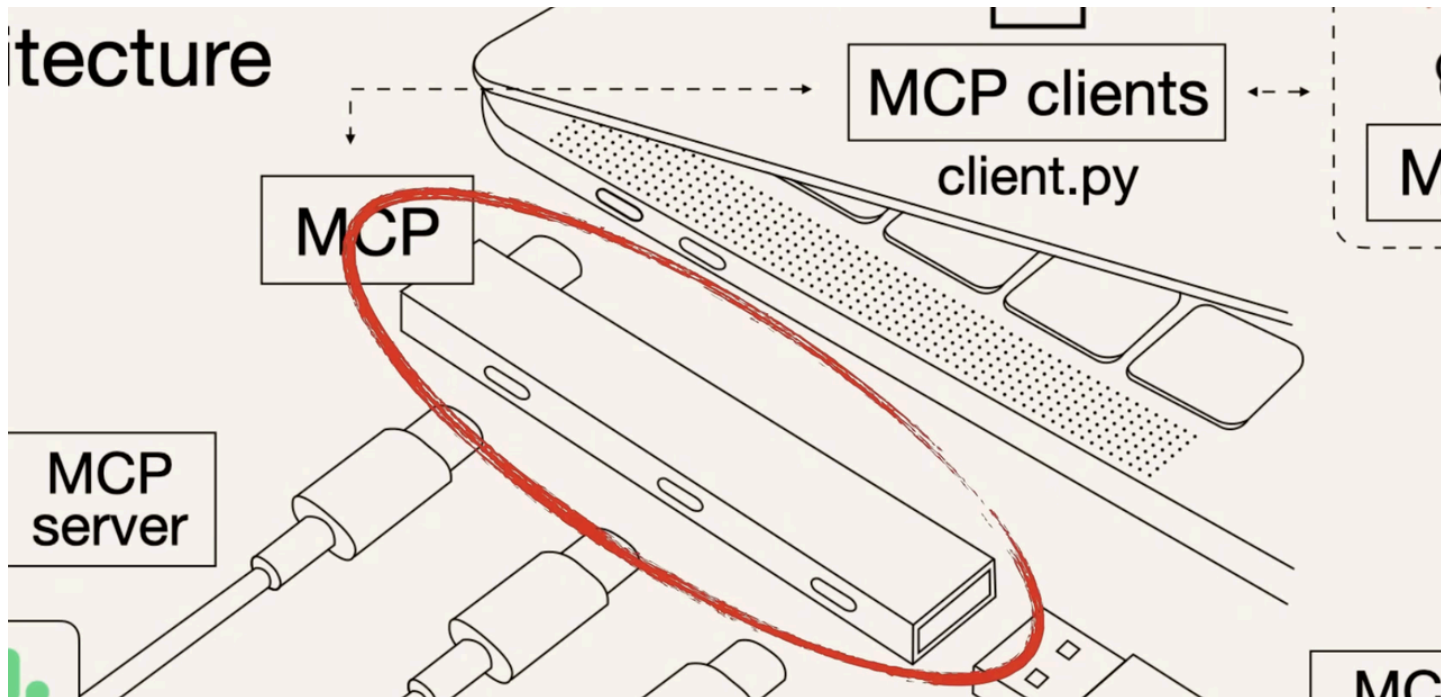
同学们好，欢迎回到 AI Agent 全栈开发课程，今天是我们智能体的最后一节课。

前面的课程，我们系统地学习了 LangChain 单智能体开发以及 LangGraph、CrewAI、AutoGen 多智能体协作框架。相信大家已经能够构建起相当复杂的智能体 Agent 系统了。

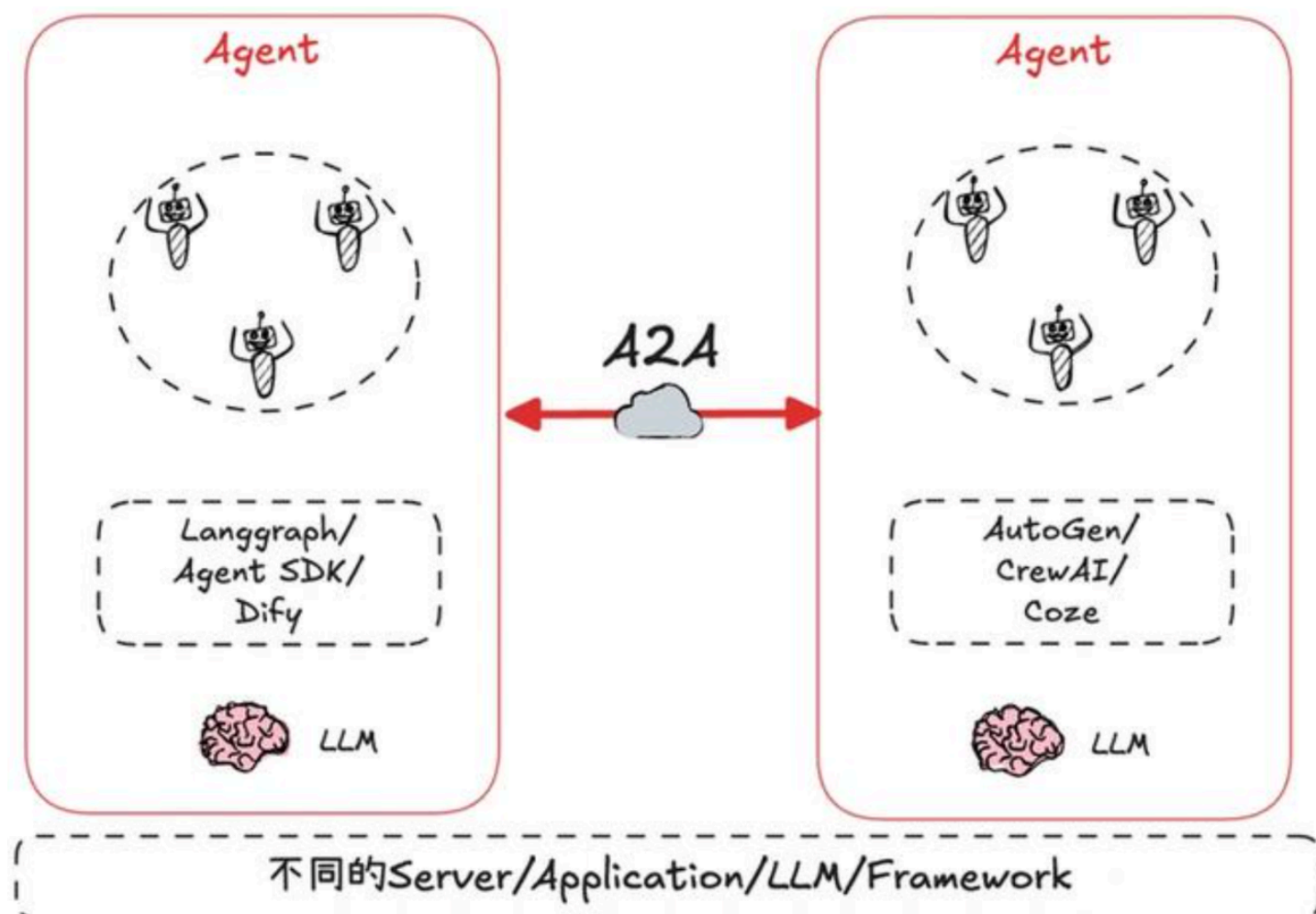
但有一个根本性问题，我们却一直没有深入讨论，那就是 **Agent 和外部世界的连接到底应该遵循什么标准？**

这一问题催生了 **MCP（Model Context Protocol，模型上下文协议）** 和 **A2A（Agent-to-Agent，智能体间通信协议）** 这两个协议的产生。时间来到 2026 年，这两套协议已成为 AI 应用基础设施的重要组成部分，其中 MCP 协议已经成为模型与工具交互的事实标准，而 A2A 协议也已被超过 150 家组织支持。

关于 `MCP` 协议：我们可以把它理解成 **AI 世界的 USB 接口**。它定义了统一的 `连接标准`，可以让任何 AI 模型都能插上任何工具。



关于 A2A 协议：我们可以把它理解为 **AI 世界的 HTTP 协议**。它定义了统一的 通信协议，让任何智能体都能和任何其他智能体协作。



今天，我们将系统性地学习这两套协议，我们介绍它们能解决什么问题、如何工作、如何集成，以及在实际项目中如何选择和使用。

我们今天的课程大纲为：

- **第一部分：**MCP 协议详解——定位、架构、核心组件
- **第二部分：**MCP Server 生命周期管理——从启动到优雅关闭
- **第三部分：**A2A 协议详解——定位、核心能力
- **第四部分：**MCP vs A2A——对比与协作
- **第五部分：**MCP 平台实操——如何创建 MCP Server、发布工具、Agent 调用
- **第六部分：**总结与作业

准备好了吗？下面我们开始。

第二部分：MCP 协议详解——定位、架构、核心组件

2.1 MCP 的诞生：解决不同模型和不同工具对接时，缺少行业标准的痛点

我们先来了解下 MCP 协议的诞生。MCP 是由 Anthropic 于 2024 年 11 月开源发布，协议的核心定位是**为大语言模型访问外部工具、数据源和服务提供标准化的接口**。简单来说，MCP 主要解决的问题是：模型跟工具对接的标准化。

在 MCP 协议出现之前，AI 行业面临一个日益严重的问题，那就是每当大模型接入外部工具时，因为市面上并没有统一的对接标准，就会出现你出一套对接风格，我出一套对接风格，由此导致开发、改造、迁移成本呈指数级增长。

于是，为了解决这个行业痛点，Anthropic 提出了 MCP。MCP 的目标是定义一个统一协议，用来取代成百上千种不同模型和工具之间混乱的点对点集成。关于 MCP，业界常用 USB 接口做类比：我们都知道 USB 接口做到了可以用一根数据线统一所有各种外设接口，只要各个外设接口都按照 USB 接口进行开发，那么它们都可以被各种不同规格的电脑进行识别。同理，MCP 的设计初衷也是为了用一个统一的协议替代各种定制化 API 工具对接方案，真正实现工具与 AI 模型之间的对接标准化，从而达到即插即用的目的。

因为 Anthropic 解决了模型和工具对接的痛点，所以 MCP 协议的接受度极高。在发布后不到四个月，OpenAI 就在 2025 年 3 月宣布在它 Agents SDK 和 ChatGPT 桌面应用中支持 MCP 协议；随后，Google DeepMind 紧随其后也将 MCP 协议整合到 Gemini 的生态系统中；微软也在 Copilot Studio 中加入了 MCP 支持。2026 年，MCP SDK 在 Python 和 TypeScript 生态中单月下载量已超过 9700 万次，数十万开发者正在基于这一标准打造和构建 AI 应用的基础设施。

2.2 核心设计原则

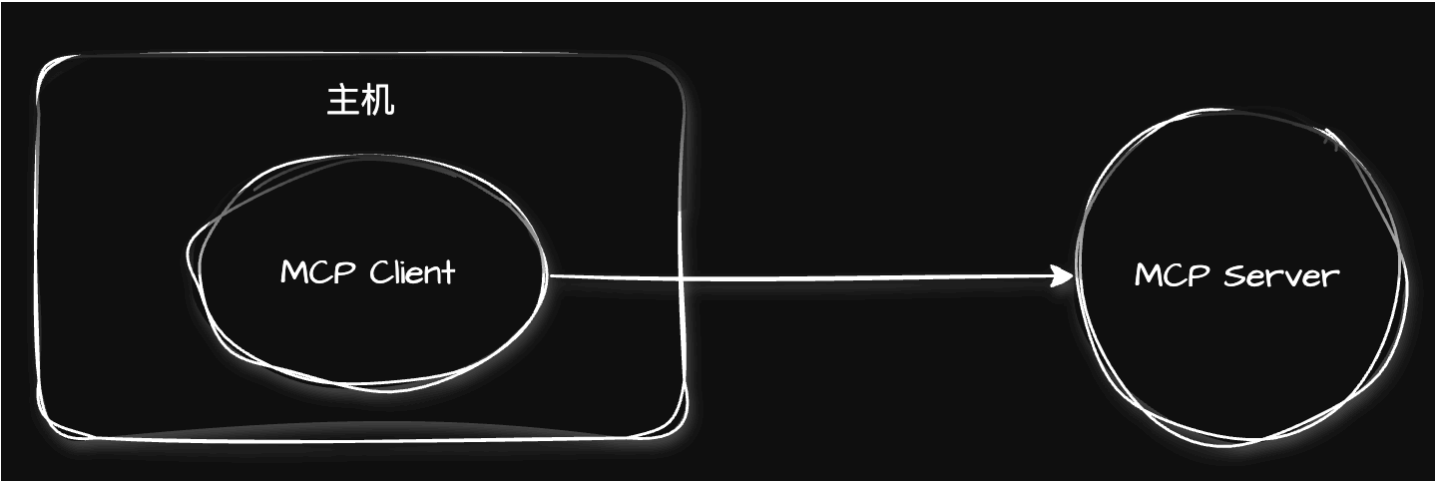
在深入介绍 MCP 的架构之前，我们有必要先理解 MCP Server 的几项核心设计原则，因为这些原则共同定义了协议的形态和边界。总结这几个协议如下：

- 第一：MCP Server（也就是工具端）应尽可能保持纯粹和简单，设计上要专注于具体、明确的功能。
- 第二：MCP Server 工具端应可以高度组合。每个 MCP Server 提供独立的功能，不同 MCP Server 之间可以无缝组合。无论 MCP 服务器是一个还是多个，MCP Client 都能用统一的交互方式调用它们。
- 第三：MCP Server 之间应做好数据隔离。它们只能处理必要的上下文信息。这是一个非常重要的安全设计。举例来说就是一个第三方的恶意天气工具不应该能读取到你和其他金融工具之间的对话历史。

2.3 客户端-主机-服务器三层架构

介绍完 MCP Server 的设计原则，下面我们介绍下 MCP 的架构。

MCP 遵循【MCP 客户端—主机—MCP 服务器】三段式技术架构。如下图所示：



MCP 客户端、主机、MCP 服务器的作用和关系为：

- **主机 (Host)**：也就是宿主机。它负责创建和管理 MCP 客户端进程，并负责管理 MCP 客户端与 MCP 服务器之间的连接。
- **客户端 (MCP Client)**：由主机所创建，用来连接 MCP 服务器，并与之通信。
- **服务器 (MCP Server)**：提供给 MCP 客户端调用，本身提供工具、数据服务等能力。在实际部署中，MCP 服务器可以部署到本地和远程，本地通过 stdio 通信，作为开发环境；也可以部署到远程，通过 HTTP 支持 MCP 客户端访问，适合生产环境部署。MCP Server 的核心职责主要是提供资源 (Resources) 和工具 (Tools) 的调用。

2.4 MCP Server 两大核心基础能力

上面介绍完 MCP 架构，下面我们重点介绍下 MCP Server 的两大核心能力，它们共同构成了 MCP 服务器向 MCP 客户端提供能力的完整体系。

第一：工具（Tools）。它是 MCP Server 中最核心的基础原子能力，也是我们实操部分重点使用的能力。工具本质上是 MCP Server 向 MCP Client 公开的可执行函数。AI Agent 可以根据用户的自然语言描述自主决定调用哪个工具、抽取什么参数。这也是 AI 智能体能够‘动手’的关键机制。

第二：资源（Resources）。它是 MCP Server 向 MCP Client 公开的只读数据源。与工具不同，资源不执行操作，只提供数据访问。资源形式上可以是文件内容、数据库查询结果以及 API 响应等等。

第三部分：MCP Server 生命周期管理

3.1 为什么需要生命周期管理

介绍完 MCP 是什么，它的三段式架构，以及 MCP Server 可以提供工具、资源这两个核心能力后，下面我们再介绍下如何管理 MCP Server 的生命周期。

MCP Server 生命周期管理从本质上来讲属于软件工程化管理范畴。

当我们在生产环境部署 MCP Server，让它提供服务时，一个必须要直面的现实是：**MCP Server 是独立进程**。它作为独立进程的好处是：

第一：即使一个 MCP Server 进程崩了，其他 MCP Server 进程仍然可用。

第二：它既可以支持跨 Agent 复用，也支持独立发版

这就是 MCP Server 作为独立进程的好处。但这些由“独立进程”带来巨大好处的同时也必然伴随着 MCP Server 需要“独立运维”的代价，那就是不同 MCP Server 都需要健康检查、沙箱隔离、故障降级等。

当然我们做 MCP Server 生命周期管理时也要分场景，不要搞绝对化。如果一个 MCP Server 只被一个智能体 Agent 在本地笔记本上使用，完全可以不做这些。但只要 MCP Server 开始被多个智能体 Agent 共享、供其他团队调用、并需要部署在服务器上长期运行，就必须要面对这些治理问题。这也是 AI 化微服务治理的自然延展。

3.2 生命周期四大阶段

MCP Server 的生命周期管理涵盖以下四个阶段：

第一阶段：启动与注册（Startup & Registration）。MCP Server 进程启动时会进行自我初始化，相关步骤如下：

1. 加载配置。比如：监听端口号、日志输出级别、工具定义等等
2. 验证所需资源。比如：数据源、外部API可用，以及完成内部能力准备。
3. 向内部 Registry（服务注册表）注册自己，上报内容包括：Server ID、版本号、提供的工具/资源列表、端点信息（stdio/HTTP/SSE）、所属风险分类、所需凭据等。

第二阶段：能力协商（Capability Negotiation）。 ~~采用JSON-RPC 2.0通信，~~MCP 客户端通过 `initialize` 请求跟 MCP Server 进行通信，MCP Server 通过响应声明自身能力（比如：可用的工具、资源等），于是双方建立能力交集，进入活动会话。

第三阶段：运行时执行与健康检查（Runtime & Health Check）。 执行期间 MCP Server 持续响应客户端的工具调用请求、管理长时任务。健康检查机制用于确保 MCP Server 持续可用：如果是 stdio Server 的生命周期由客户端进程管理，超时或意外退出即视为不健康；HTTP Server 需提供独立的 `/health` 端点，供负载均衡器或监控系统定期探测。

第四阶段：优雅停用（Graceful Shutdown & Retirement）。 当 MCP Server 需要下线或更新版本时，Stop 指令发出后，MCP Server 完成以下步骤：

1. 停止接收新请求
2. 处理完进行中的请求
3. 关闭与客户端的连接
4. 向服务注册表（Registry）注销自身
5. 释放资源（文件句柄、数据库连接等）
6. 退出

3.3 生产环境治理要点

MCP Server 进入生产环境后，需要关注以下治理要点。

1. **安全与凭据管理。** 凭据不应写在 `.env` 文件随代码上传，也不应硬编码或存放在不可信路径。必须使用密钥管理系统（KMS）或通过 Registry 下发。这里没事下凭据到底指的是什么。所谓凭据，是指MCP Server 访问外部资源（数据库、第三方API、内部服务、文件系统等）所需的敏感认证信息。
2. **沙箱强制隔离。** MCP Server 默认不应完全信任。给 MCP Server 访问的文件系统、网络请求、命令执行都应在受限环境中进行，例如 Docker 容器、只读文件系统等。这能有效防止恶意工具带来的供应链攻击。
3. **结构化错误与故障降级。** MCP Server 不应将所有异常向智能体 Agent 透传。智能体 Agent 收到大段堆栈跟踪后可能理解偏差、泄露内部信息或陷入重试死循环。正确做法是：Agent 实现超时控制和重试上限，MCP Server 将系统级错误翻译为对 Agent 友好的错误消息；降级策略包括从实时数据切换到缓存数据、从精确计算切换到近似估算、从完整服务降级到核心服务等。

4. **可观测性。** MCP Server 是企业 AI 基础设施的一部分，应接入统一监控体系：

- a. 每个工具调用记录耗时、成功/失败状态、参数摘要
- b. 结构化日志输出到日志平台
- c. 关键指标暴露到 Prometheus

第四部分：MCP 生态与 2026 年演进

4.1 生态全景

MCP 生态在 2025-2026 年间经历了爆发式增长。

1. **核心基础设施层面**，截至到现在，MCP SDK 已积累超过 1.5 亿次下载，支持 TypeScript 和 Python 等多种语言，覆盖了主流 AI 开发工具链。
2. **AI 平台深度集成。** 微软的VS Code、Cline 扩展、Continue 插件，Cursor 编辑器，Windsurf IDE，以及Claude Code 和 Gemini-CLI，都已内建 MCP 支持。开发者只需要一个 MCP Server 定义，就能让所有这些 AI 工具同时获得调用能力。
3. **社区贡献呈指数级增长。** 数千个 MCP Server 被发布到社区 Registry 和各类市场，能力包括覆盖 GitHub 文件操作、Slack 消息收发、PostgreSQL 数据库查询、Google Drive 文档读写、Jira 工单创建等几乎所有主流开发场景。这里重点推荐：
 - a. [modelcontextprotocol/servers](#)。这是官方维护的 MCP Server 集合
 - b. [awesome-mcp-servers](#)。这是社区精选列表，收录了很多高质量 MCP Server这些都是我们日常工作和学习时，比较有帮助的 MCP Server 首选参考源。

第五部分：A2A协议详解——智能体间通信

5.1 A2A 的定位与核心能力

A2A (Agent-to-Agent) 协议是 Google 于 2025 年 4 月 9 日发布的开放标准，旨在为不同生态系统中的 AI 智能体提供安全、标准化的协作框架。

A2A 协议要解决的核心问题是：Salesforce（全球领先的 CRM 平台）的 Agent 如何将任务委派给 ServiceNow（企业数字化 workflows 平台）的 Agent？Google Vertex（Google Cloud 的统一 AI/ML 开发平台）的 Agent 如何与 AWS Bedrock（Amazon 的托管式基础模型服务平台）的 Agent 协同？此前，每一个跨厂商的 Agent 协作因为缺乏标准都需要进行定制化集成，而 A2A 的出现，开启了使用统一开放标准替代不同 Agent 供应商点对点的定制集成。就像不同国家、不同语种的人可以通过国际通用语——英语一样，A2A 就是智能体 Agent 世界的英语。

A2A 协议定义了三个基本规范：

1. Agent Cards（能力通告）
2. Tasks（任务交换）
3. Transport（传输协议）。

其中 Agent Cards 是位于 `/.well-known/agent-card.json` 的标准化的 JSON 文件，发布后其他 Agent 可读取该 Card 并按能力发现。Tasks 定义了标准化的任务生命周期，包括：已提交 → 执行中 → 需补充输入 → 已完成/失败/已取消/已拒绝。是独立于具体业务领域的通用协作协议。传输协议主要基于：HTTP + SSE + JSON-RPC 2.0，以现有 Web 技术栈实现互通，无需新协议或专用网关，支持同步（task/send）和流式（task/sendSubscribe）两种任务模式，为长时任务提供实时进度推送。

A2A 项目在启动时已有超过 50 家合作伙伴，2025 年 6 月贡献给 Linux 基金会并采用 Apache 2.0 许可证开源。到 2026 年 4 月，支持组织已增长至超过 150 家，涵盖 AWS、Cisco、Google、IBM、Microsoft、Salesforce、SAP 和 ServiceNow 等核心企业。

5.2 典型工作流程

A2A 协议的工作流程通过四步标准化的消息交换完成，整个过程中 Agent 双方无需暴露私有代码或内部数据。

第一步：发现能力（Discovery）。 Client Agent 请求读取 Server Agent 发布的 Agent Card，获取其技能列表、端点地址和支持的认证方式。通过 `/.well-known/agent-card.json` 标准化路径实现自动发现。

第二步：认证授权（Authentication）。 Client Agent 根据 Agent Card 声明的认证方式（通常基于 OAuth 2.0/OIDC）获取短期有效令牌（生命期按分钟计算），用于后续请求的身份验证。

第三步：任务委托（Task Delegation）。 Client Agent 发起任务请求（`task/send` 同步等待或 `task/sendSubscribe` 流式接收进度）。Server Agent 处理请求并返回结构化响应。

第四步：可观测性（Observability）。 整个通信链路携带分布式追踪 ID，Agent 结构化日志以 OTLP 格式导出，实现端到端调用的完整可观测性。

第六部分：MCP vs A2A——对比与协作

6.1 核心差异与定位分工

MCP 和 A2A 协议各自定位在智能体通信栈的不同层次，两者共同构建完整的智能体协作协议体系。

MCP 关注智能体与外部工具的交互（Agent-to-Tools），解决的是模型如何统一访问数据库、API、文件系统等资源的问题；A2A 关注智能体之间的通信与协作（Agent-to-Agent），解决的是多个智能体如何互相发现、委派任务、协调工作的问题。

在技术栈层次上，MCP 运行在连接层，是‘纵向集成’协议，连接应用层和工具资源层；A2A 运行在协作层，是‘横向集成’协议，连接不同智能体。在通信模式上，MCP 遵循客户端-服务器模型（一个 Agent 作为客户端调用一个 Server 工具），A2A 遵循对等模型，多个 Agent 在对等网络中直接通信。在消息格式上，MCP 和 A2A 都基于 JSON-RPC 2.0，但 MCP 主要用于工具调用和资源访问，A2A 的 Agent Cards 使用 JSON Schema 描述能力，支持通过 Registry 进行更精细的服务发现。

实际生产部署中，MCP 和 A2A 往往是配合使用的。一个典型的数据分析工作流会使用 A2A 协议进行 Agent 级路由分发，将不同用户请求路由到相应的 Agent 实例，然后 Agent 内部再通过 MCP 协议执行具体的工具调用——数据查询、文件读写、模型推理等。两者分工明确，互不重叠。

6.2 选型框架与协作策略

理解了 MCP 和 A2A 的定位差异后，接下来技术选择就自然变得清晰了。

需要给单个 Agent 提供标准化的工具调用接口的场景，我们选择 MCP 协议。

而需要让多个 Agent 协同完成复杂工作流，我们使用 A2A 协议。

这里需注意，MCP 和 A2A 这两个协议在协作策略上是互补而非互斥的。一个大型企业通常同时采用 MCP 和 A2A 两种协议来构建端到端的 Agent 基础设施：Agent 内部用 MCP 标准化工具接入能力；Agent 间的协作通过 A2A 协议进行任务委派与协调；跨企业的 Agent 交互叠加 ASL 协议确保身份可信。可以形象地理解：~~MCP=USB-C（统一工具接口），A2A=HTTP+JSON-RPC（统一通信协议），ASL=TLS+OAuth（统一安全信任层）。~~

从企业内部的技术演进节奏来看，先以 MCP 快速构建工具接入，然后在单 Agent 能力成熟后自然演进到以 A2A 驱动的多 Agent 动态交互。两者形成的正是 AI 原生基础设施的完整分层架构。

第七部分：MCP 平台实操

7.1 搭建基础 MCP Server

现在我们进入今天的实战环节——从头构建一个完整的 MCP Server，并让 AI Agent 能通过标准化协议调用我们编写的工具。

步骤1：环境准备

代码块

```
1  mkdir mcp
2  cd mcp
3  python -m venv venv
4  source venv/bin/activate
5  pip install mcp httpx python-dotenv
```

步骤2：创建基础 Server（stdio模式）

接着搭建 MCP Server，这里我们采用 stdio（标准输入输出）模式，也就是本地单机标准输入输出模式。

我们创建 `server.py`，写入以下内容：

代码块

```
1  import asyncio
2  import json
3  # MCP Server 核心框架
4  from mcp.server import Server, NotificationOptions
5  from mcp.server.models import InitializationOptions
6  # stdio 传输适配器, 让 Server 通过 stdin/stdout 与客户端通信
7  import mcp.server.stdio
8  # MCP 协议的数据类型定义 (Tool、TextContent 等)
9  import mcp.types as types
10
11 # 创建 MCP Server 实例, 初始化了一个 MCP Server
12 server = Server("mcp-stdio-server")
13
14 # 简单的内存字典, 模拟股票数据。实际场景可替换为真实股票 API 调用。
15 STOCK_DB = {
16     "AAPL": {"price": 175.50, "change": "+2.3%", "pe": 28.5},
17     "GOOGL": {"price": 142.80, "change": "+1.2%", "pe": 25.1},
18     "MSFT": {"price": 420.30, "change": "+0.8%", "pe": 35.2}
19 }
20
21 # 这是 MCP 协议的核心端点之一。
22 # 当 AI 客户端询问"你能做什么"时, MCP Server 返回所有可用工具的定义。
23 @server.list_tools()
24 async def handle_list_tools() -> list[types.Tool]:
25     """返回 Server 支持的所有工具列表"""
26     return [
27         types.Tool(
28             name="get_stock_price",
29             description="获取指定股票的当前价格和基本信息。参数 symbol 是股票代码, 如 'AAPL'、'GOOGL'。",
30             # inputSchema 使用 JSON Schema 标准, 让 AI 知道该传什么参数
31             inputSchema = {
32                 "type": "object",
33                 "properties": {
34                     "symbol": {"type": "string", "description": "股票代码"}
35                 },
36                 "required": ["symbol"]
37             }
38         ),
39         types.Tool(
40             name="calculate_bmi",
41             description="根据体重 (公斤) 和身高 (米) 计算BMI指数并给出健康建议。",
42             inputSchema={
43                 "type": "object",
44                 "properties": {
45                     "weight": {"type": "number", "description": "体重 (公斤) "},
46                     "height": {"type": "number", "description": "身高 (米) "}
```

```

47         },
48         "required": ["weight", "height"]
49     }
50 )
51 ]
52
53 # 这是另一个核心端点。
54 # 当 AI 决定要调用某个工具时，客户端发送函数 name + 函数 arguments，
55 # 然后 MCP Server 执行并返回结果。
56 @server.call_tool()
57 async def handle_call_tool(
58     name: str, arguments: dict | None
59 ) -> list[types.TextContent | types.ImageContent | types.EmbeddedResource]:
60     """执行工具调用"""
61     if name == "get_stock_price":
62         symbol = arguments.get("symbol", "").upper()
63         if symbol not in STOCK_DB:
64             return [types.TextContent(type="text", text=f"未找到股票代码
65 {symbol}")]
66         data = STOCK_DB[symbol]
67         return [types.TextContent(
68             type="text",
69             text=f"{symbol} 当前价格: ${data['price']}, 涨跌幅:
70 {data['change']}, PE: {data['pe']}"
71         )]
72     elif name == "calculate_bmi":
73         weight = arguments.get("weight")
74         height = arguments.get("height")
75         if not weight or not height or height <= 0:
76             return [types.TextContent(type="text", text="无效的体重或身高")]
77         bmi = weight / (height ** 2)
78         if bmi < 18.5:
79             category = "偏瘦"
80         elif bmi < 25:
81             category = "正常"
82         elif bmi < 30:
83             category = "超重"
84         else:
85             category = "肥胖"
86         return [types.TextContent(
87             type="text",
88             text=f"BMI指数: {bmi:.1f}, 分类: {category}"
89         )]
90     else:
91         raise ValueError(f"未知工具: {name}")
92
93 async def main():

```

```

92     async with mcp.server.stdio.stdio_server() as (read_stream, write_stream):
93         await server.run(
94             read_stream,
95             write_stream,
96             InitializationOptions(
97                 server_name="mcp-stdio-server",
98                 server_version="0.1.0",
99                 capabilities=types.ServerCapabilities(
100                     tools=types.ToolsCapability(listChanged=False)
101                 )
102             )
103         )
104
105 if __name__ == "__main__":
106     asyncio.run(main())

```

这段代码实现了一个基于 stdio 传输的 MCP Server，它提供两个工具（Tool）给 AI 调用：

1. `get_stock_price` — 查询股票价格
2. `calculate_bmi` — 计算 BMI 指数

AI 客户端通过标准输入输出接口与本程序进行通信。

核心代码解析：

- `@server.list_tools()`：在 MCP 客户端连接时被调用，返回 MCP Server 支持的所有工具的名称、描述和参数 Schema。Agent 通过读取这些工具列表决定可调用哪些工具。该接口告诉智能体 Agent，我有什么，能做什么。
- `@server.call_tool()`：接收工具名和参数，执行具体业务逻辑，返回结构化结果。该接口告诉智能体 Agent，给我派活，我爱工作。
- `mcp.server.stdio.stdio_server()`：通过标准输入输出与 MCP 客户端进行通信。

7.2 测试 MCP Server

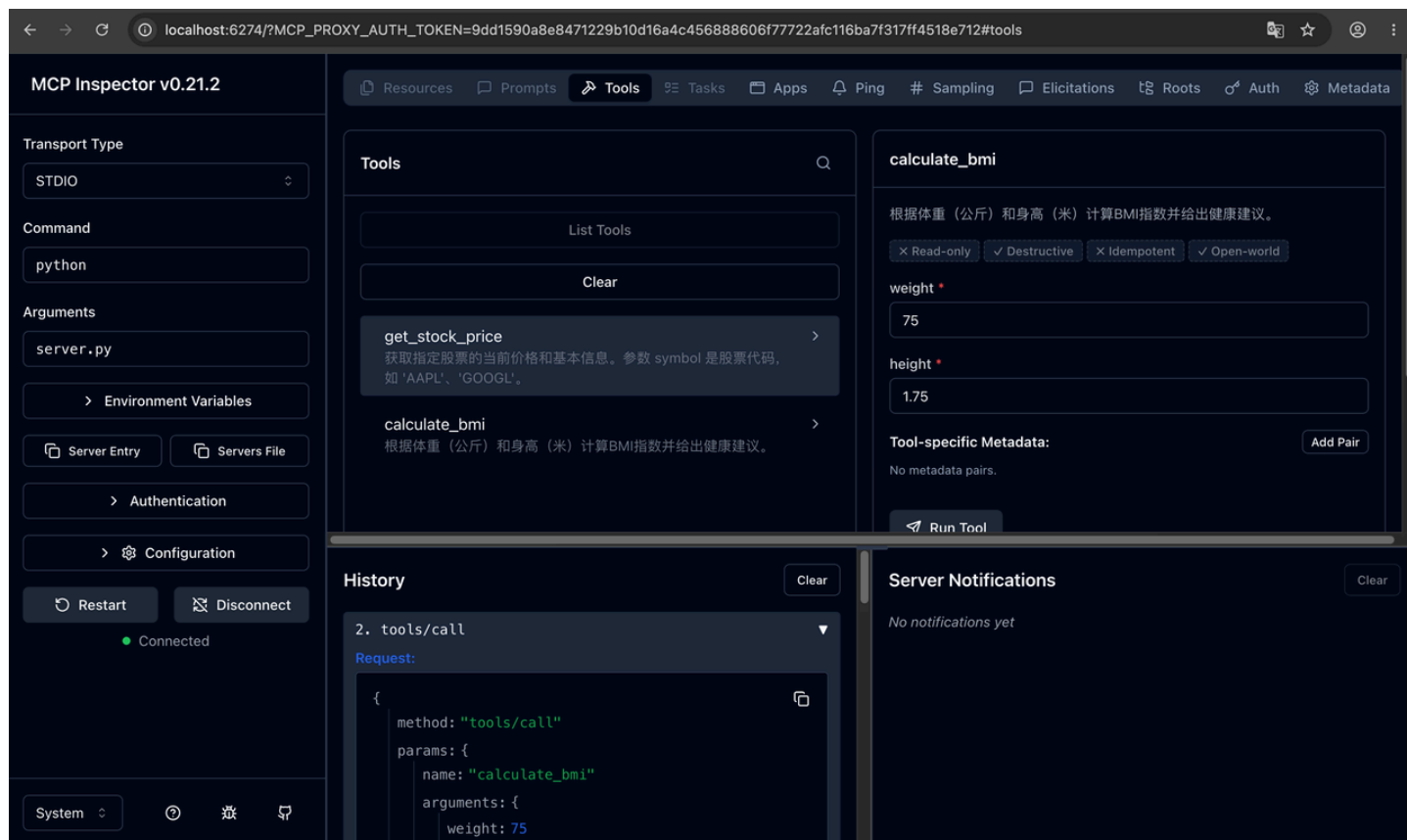
MCP Server 写好后，我们通过 MCP Inspector 进行测试，这是一个官方提供的可视化调试工具。

7.2.1 启动 MCP Inspector

代码块


```
1 npx @modelcontextprotocol/inspector python server.py
```

Inspector 会在浏览器中打开调试界面。测试 `get_stock_price` 工具时，输入参数 `{"symbol": "AAPL"}`，返回结果为 `AAPL 当前价格: $175.50, 涨跌幅: +2.3%, PE: 28.5`。MCP Inspector 会显示完整的 JSON-RPC 请求和响应内容，便于我们理解协议底层的消息格式。执行结果如下图所示：



7.2.2 通过 MCP Client 调用

我们还可以通过编程方式，手写一个 MCP Client 调用 MCP Server 来验证功能。

我们编写 `client.py` 文件：

代码块

```
1 import asyncio
2 from mcp import ClientSession, StdioServerParameters
3 from mcp.client.stdio import stdio_client
4
5 async def run():
6     server_params = StdioServerParameters(command="python", args=["server.py"])
7     async with stdio_client(server_params) as (read, write):
8         async with ClientSession(read, write) as session:
```

```

9         await session.initialize()
10        # 列出可用工具
11        tools = await session.list_tools()
12        print("可用工具:", [t.name for t in tools.tools])
13        # 调用工具
14        result = await session.call_tool("get_stock_price", arguments=
{"symbol": "AAPL"})
15        print("结果:", result.content[0].text)
16
17    if __name__ == "__main__":
18        asyncio.run(run())

```

执行结果如下图所示：

```

[(venv) ai@aideMacBook 8-mcp % python client.py
可用工具：['get_stock_price', 'calculate_bmi']
结果：AAPL 当前价格：$175.5，涨跌幅：+2.3%，PE：28.5

```

7.3 将 MCP Server 接入 AI Agent

上面我们演示了一个 MCP 客户端调用 MCP 服务端的项目用例。

下面我们实现把 MCP Server 接入到真实的 AI 智能体 Agent。

将 MCP Server 集成到我们的智能体 Agent 非常简单，使用 OpenAI Agents SDK 提供的 MCP 集成，可以轻松通过几行代码就完成工具的接入，实现方式如下：

代码块

```

1  import os
2  from dotenv import load_dotenv
3
4  # 加载 .env 文件（必须在导入 agents 之前）
5  load_dotenv()
6  os.environ["OPENAI_AGENTS_DISABLE_TRACING"] = "1" # 关闭追踪
7
8  # 验证是否加载成功
9  if not os.getenv("OPENAI_API_KEY"):
10      raise ValueError("OPENAI_API_KEY not found in .env file")
11
12  import asyncio
13  from agents import Agent, Runner
14  from agents.mcp import MCPServerStdio

```

```
15
16  async def run_agent():
17      async with MCPServerStdio(
18          name="Demo MCP Server",
19          params={"command": "python", "args": ["server.py"]}
20      ) as server:
21          agent = Agent(
22              name="Assistant",
23              instructions="You are a helpful assistant.",
24              mcp_servers=[server]
25          )
26          result = await Runner.run(agent, "查一下苹果公司股票的当前价格")
27          print(result.final_output)
28
29  if __name__ == "__main__":
30      asyncio.run(run_agent())
```

执行该 agents.py 代码时，需要先安装 openai agents sdk 依赖，执行如下语句：

代码块

```
1  pip install openai-agents
```

这里声明下：

我们这里演示的 stdio 本地模式只适合开发和单机部署；生产环境我们还是强烈推荐将 MCP Server 封装为 HTTP/SSE 服务，并部署进容器平台，比如 Docker 或者 k8s；然后通过引入 API 网关实现请求调用的鉴权、限流和路由能力；并通过 Registry 进行 MCP Server 的服务发现和版本管理；最后接入监控、告警和可观测性。

第八部分：课程总结与作业

8.1 核心要点回顾

今天的内容是AI Agent工程化的重要里程碑。我们来快速回顾：

技术层面：

- **✓ MCP协议**：AI世界的USB-C标准，统一模型与工具的交互，三层架构（主机-客户端-服务器），三大原语（工具、资源、提示）。
- **✓ MCP Server生命周期**：启动注册、能力协商、运行时执行、优雅停用，安全治理要点包括凭据隔离和沙箱强制执行。
- **✓ A2A协议**：智能体间的通用语言，Agent Cards 能力发现、Tasks 标准化任务生命周期、HTTP+SSE+JSON-RPC 2.0 传输。
- **✓ MCP vs A2A**：MCP负责 Agent-to-Tools 纵向集成，A2A负责 Agent-to-Agent 横向协作，两者配合构建完整智能体基础设施。选型遵循‘单一 Agent 接入工具选 MCP，多 Agent 跨平台协作需增加 A2A’的原则。

能力层面：

- **✓** 能够从零构建 MCP Server，定义工具并暴露给 Agent
- **✓** 能够通过 MCP Client 编程方式调用 Server
- **✓** 能够理解 MCP Server 生命周期和生产环境治理要点
- **✓** 能够根据场景做出 MCP/A2A 的技术选型
- **✓** 理解智能体通信协议的完整生态

8.2 课后作业

1. **MCP Server实战**：实现一个包含至少3个工具的自定义MCP Server（工具可包括天气查询、汇率转换、新闻搜索等）。
2. **生命周期设计**：为作业1的 Server 编写一份生产环境部署方案文档（Markdown格式），涵盖健康检查设计、凭据管理方案、沙箱隔离措施、优雅停用策略。
3. **协议选型分析**：给定一个‘企业级多智能体数据分析平台’需求，分析哪些组件需要 MCP、哪些需要 A2A，画出整体架构图，并撰写选型理由。

附录：教学资源

项目结构与关键文件

```
代码块
1  mcp_demo_server/
```

2	—	server.py	# MCP Server实现
3	—	client.py	# MCP Client调用示例
4	—	requirements.txt	
5	—	.env	

关键命令速查

操作	命令
启动 MCP Inspector	<code>npx @modelcontextprotocol/inspector python server.py</code>
安装 MCP SDK	<code>pip install mcp</code>
安装 MCP Inspector	<code>npm install -g @modelcontextprotocol/inspector</code>
测试 A2A Agent Card	<code>curl http://agent-endpoint/.well-known/agent-card.json</code>

推荐阅读

- [MCP官方文档](#)
- [MCP 2026-07-28 Release Candidate公告](#)
- [A2A官方规范（GitHub）](#)
- [中国信通院：智能体安全可信互连协议（ASL）](#)
- **官方Server集合：** github.com/modelcontextprotocol/servers
- **社区精选Server列表：** github.com/punkpeye/awesome-mcp-servers
- **MCP Inspector：** `npx @modelcontextprotocol/inspector`